

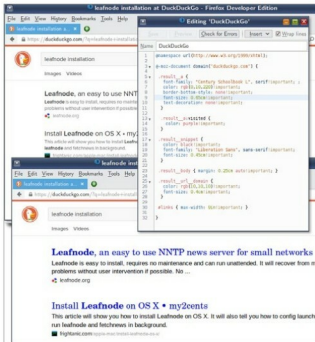


Figure 4: These screenshots show the 'before' and 'after' appearances of a DuckDuckGo search engine results page. The actual stylesheet code is not important as the scope and possibilities that these screenshots illustrate

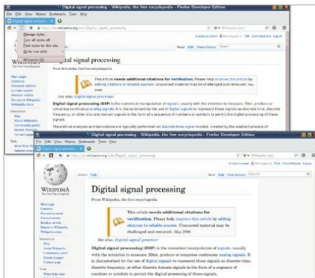


Figure 5: The menu of the Stylis add-on will allow you to install not only your stylesheets but also those published by others on the userstyles.org website

```
// @description Get past the page that asks for your phone
number
// @include      https://edit.yahoo.com/progreq/*
// @exclude     %exclude%
// @version     1
// @grant       none
// ==/UserScript==

document.addEventListener("DOMContentLoaded",
```

```
getPastPhoneNumberPage,
false);
```

```
function getPastPhoneNumberPage() {
  try {
    //window.alert("Moving past the page");
    location.href = "https://mail.yahoo.com";
  } catch (err) {
    console.log("BPNPYM Greasemonkey Error: " + err);
  }
}
```

If you refused to provide your number once, then should not the site respect that decision by remembering it? By playing dumb every time you log in, it is trying to coerce you into parting with your number.

User style examples

Figure 4 shows a DuckDuckGo page. I use this search engine as the default because it has better respect for people. However, I found that the colours and the contrast on its Web pages were not much to my liking. So, I wrote a user CSS for it, remembering to suffix the styles with an 'important' attribute to override the page authors' setting.

For the Wikipedia website, I use the following CSS because its pages are usually text-heavy and take up the entire width of the screen. On big monitors, they can literally be a pain in the neck.

```
* { font-family: Century Schoolbook !important; }
div#content { Font-size: 1.2em; width: 71%; }
```

User JS/CSS on the mobile

Being a fan of user scripts and styles, I added the ability to run them in a namesake browser that I created for Android. While the user styles can be written in the usual way, the user scripts have to be written in bookmarklet format, prefixed with 'javascript'. For example, to edit a Web page, I use this script:

```
javascript:(function() { document.body.
contentEditable=true;document.designMode='on'; }) ());
```

Once executed, this brings up the on-screen keyboard and you can then start editing the current Web page. The same code works in a regular desktop browser too. Go ahead and type it in your address bar.

Maintenance

Of course, Web pages change and the IDs (or whatever you had tried to meddle with) change along with them. Your Greasemonkey script may eventually stop working. You will have to either fix it yourself, update it from the Web or find a new one.

Recently, I published a Greasemonkey script that would